

Relatório do Projeto

Projeto de Compiladores 2025/26

UNIDADE CURRICULAR: Compiladores

ALUNO: João Natálio

Nº ESTUDANTE: 2023205576

DATA: Maio de 2026

Resumo

*Este relatório documenta o desenvolvimento de um compilador completo para a linguagem Juc, um subconjunto de Java. O compilador foi implementado em C com recurso às ferramentas *lex* e *yacc*, e está estruturado em quatro componentes encadeados: analisador léxico, analisador sintático, analisador semântico e gerador de código LLVM IR. São descritos os algoritmos, estruturas de dados e decisões de implementação relevantes em cada fase.*

1. Gramática Re-escrita

1.1 Análise Léxica (Meta 1)

O analisador léxico foi implementado em *lex* no ficheiro **jucompiler.l**. A contagem de linha e coluna é feita centralmente na macro **YY_USER_ACTION**, que guarda os valores correntes antes de cada acção, garantindo que os erros sejam reportados na posição de início de cada token.

Foram definidas duas condições de início exclusivas — **COMMENT** e **STR** — para tratar comentários multi-linha e literais de cadeia de caracteres, respectivamente. Dentro de **STR**, as sequências de escape válidas (`\n`, `\t`, `\r`, `\f`, `\\`, `\\\`) são acumuladas num buffer; sequências inválidas geram o erro correspondente e activam uma flag que suprime a emissão do token **STRLIT**, mas prossegue a análise até à aspa de fecho, reportando depois o erro de *unterminated string* apenas se necessário.

Dois macros auxiliares simplificam a emissão de tokens: **RET(tok, text)** para tokens sem valor semântico e **RET_VAL(tok, text, val)** para tokens com valor. Ambos preenchem a estrutura **yyval.token** com a string, linha e coluna do token antes de o devolver ao parser, assegurando que a informação de localização flui até à AST.

1.2 Transformação da Gramática EBNF para YACC

A gramática original é inerentemente ambígua e incompatível com a análise LR(1). A transformação exigiu três tipos de intervenção: eliminação das repetições **{...}** por recursividade à esquerda; desambiguação dos operadores por declarações de precedência; e resolução do conflito *dangling else*.

Regras recursivas à esquerda.

As repetições em **DeclList**, **FormalParamsList**, **MethodBodyDecls** e **BlockStatements** foram convertidas em regras recursivas à esquerda, evitando o crescimento ilimitado da pilha do parser e garantindo associatividade correcta.

```
DeclList : /* vazio */
        | DeclList MethodDecl
        | DeclList FieldDecl
        | DeclList SEMICOLON
```

Declarações de precedência e associatividade.

Os operadores são declarados por ordem crescente de precedência, alinhados com as convenções de Java. A tabela seguinte resume as decisões tomadas:

Nível	Operadores	Associatividades
1	=	Direita
2		Esquerda
3	&&	Esquerda
4	^	Esquerda
5	==, !=	Esquerda
6	<, <=, >, >=	Esquerda
7	<<, >>	Esquerda
8	+, -	Esquerda
9	*, /, %	Esquerda
10	!, - (unário)	Direita

Tabela 1 — Precedências e associatividades dos operadores em Juc.

Conflito dangling else.

O conflito clássico de pendência do **else** foi resolvido declarando **%nonassoc THEN** com precedência inferior a **%nonassoc ELSE**, e anotando a regra sem **else** com **%prec THEN**. Desta forma, o yacc associa sempre o **else** ao **if** mais próximo, em conformidade com a semântica de Java.

```
Statement: IF LPAR Expr RPAR Statement %prec THEN { ... }
          | IF LPAR Expr RPAR Statement ELSE Statement { ... }
```

Separação Expr / BinaryExpr.

A gramática define uma regra **Expr** que abrange **Assignment** e **BinaryExpr** como alternativas de topo, mantendo a atribuição semanticamente distinta e impedindo cascatas inválidas como **a = b = c**, que não existem em Juc.

Recuperação de erros.

Foram incluídas cinco regras de recuperação de erros, limitando a propagação a unidades sintáticas coesas e permitindo que o parser retorne a análise após cada erro:

```
FieldDecl      : error SEMICOLON { $$ = NULL; }
Statement      : error SEMICOLON { $$ = NULL; }
MethodInvocation: IDENTIFIER LPAR error RPAR { $$ = NULL; }
ParseArgs      : PARSEINT LPAR error RPAR { $$ = NULL; }
Expr           : LPAR error RPAR { $$ = NULL; }
```

A função **yyerror** acede às variáveis globais **syn_line**, **syn_column** e **error_yytext**, preenchidas pela macro **YY_USER_ACTION** do lexer, emitindo a mensagem de erro com localização exacta.

2. AST e Tabela de Símbolos

2.1 Estrutura de Dados da AST

A Árvore de Sintaxe Abstrata é representada por um grafo de nós heterogêneos com filhos em lista ligada. Cada nó contém a categoria (enum), o valor lexical, o tipo semântico, uma anotação opcional de assinatura, e informação de localização:

```
struct node {
    enum category    category;    // Tipo do nó (ProgramNode, AddNode, ...)
    char             *token;      // Valor lexical (NULL se não aplicável)
    int              semantic_type; // Tipo semântico (TY_INT, TY_BOOL, ...)
    const char       *annot;      // Assinatura de método, ex: "(int,int)"
    int              line, col;    // Localização no ficheiro fonte
    struct node_list *children;    // Lista ligada de filhos
};
```

O primeiro elemento de **children** é um nó sentinela com **node == NULL**, que simplifica a inserção sem verificações especiais de lista vazia. O campo **annot** foi adicionado para anotar nós **Identifier** correspondentes a invocações de métodos com a assinatura formal, por exemplo **(int,int)**, necessária para a impressão correcta da AST anotada.

A gestão de nós redundantes é feita em construção: um **Block** com zero statements converte-se em **NullNode**; um bloco com exatamente um statement devolve esse statement directamente, sem envolvente; dois ou mais statements mantêm o nó **Block**. Os nós **Program**, **MethodParams** e **MethodBody** nunca são considerados redundantes, independentemente do número de filhos.

2.2 Estrutura de Dados da Tabela de Símbolos

A tabela de símbolos é organizada em duas camadas: uma tabela global por classe e uma tabela local por método. As tabelas estão ligadas em lista (**SymbolTable *next**) e cada tabela mantém uma lista ligada de símbolos (**Symbol *first**):

```
typedef struct Symbol {
    char          *name;
    SymbolType     type;
    ParamList      *param_types; // Tipos dos parâmetros (métodos)
    int            is_param;      // É parâmetro formal?
    int            is_method;     // É método?
    struct SymbolTable *method_table;
    struct Symbol  *next;
} Symbol;

typedef struct SymbolTable {
    char          *name;
    ParamList      *method_params;
    Symbol         *first;
    struct SymbolTable *next;
} SymbolTable;
```

O tipo **ParamList** é uma lista ligada de **SymbolType**, usada tanto para representar os parâmetros formais de um símbolo de método como para construir a lista de tipos dos argumentos numa invocação. A sobrecarga de métodos é suportada naturalmente: a chave de pesquisa é o par (nome,

lista de tipos de parâmetros).

2.3 Algoritmo de Construção das Tabelas (check_program)

A análise semântica decorre em **duas passagens** sobre a AST, implementadas em **check_program**:

1.ª passagem — declarações de topo. Itera sobre os filhos directos do nó **Program**. Para cada **FieldDecl**, insere a variável global na tabela global, verificando duplicados e a palavra reservada `_`. Para cada **MethodDecl**, constrói a **ParamList** dos parâmetros formais com uma tabela temporária para detecção de parâmetros duplicados dentro da mesma assinatura, verifica duplicados de assinatura (suporte a sobrecarga), e insere o símbolo de método na tabela global.

2.ª passagem — corpos dos métodos. Para cada **MethodDecl**, cria a tabela local, insere o símbolo especial **return** com o tipo de retorno declarado, insere os parâmetros formais válidos, e invoca **annotate_ast** sobre o nó **MethodBody**, passando a tabela local como contexto.

2.4 Algoritmo de Verificação de Tipos (annotate_ast)

annotate_ast é um percurso **pós-ordem** recursivo da AST. Cada nó é visitado após todos os seus filhos, garantindo que os tipos dos operandos estão disponíveis quando o nó pai é processado. O resultado de cada nó é guardado em **semantic_type**. As principais regras de tipagem são:

Nó	Tipo resultante / Regra
Natural	TY_INT; erro se valor > 2 147 483 647
Decimal	TY_DOUBLE; erro se overflow ou underflow para zero
BoolLit	TY_BOOL
Identifier	Tipo do símbolo; pesquisa local → global; erro Cannot find symbol
+, -, *, /, %	TY_DOUBLE se algum operando for double, TY_INT caso contrário; proibido para bool e String[]
<, <=, >, >=	TY_BOOL; operandos numéricos
==, !=	TY_BOOL; operandos do mesmo grupo (numérico ou bool)
&&,	TY_BOOL; ambos os operandos devem ser bool
^, <<, >>	TY_INT; ambos os operandos devem ser int
return	Verificação de compatibilidade com o tipo de return na tabela local; int → double permitido
if / while	Condição deve ser bool
System.out.print	Proibido para void, undef, String[]

Tabela 2 — Regras de tipagem implementadas em **annotate_ast**.

Resolução de sobrecarga em CallNode.

Para cada invocação de método, o algoritmo percorre a tabela global contabilizando separadamente correspondências **exactas** (tipos idênticos) e **compatíveis** (onde **int** pode ser promovido para **double**). A selecção segue a hierarquia:

Situação	Acção
Existe correspondência exacta	Seleccionar esse método
Existe exactamente uma correspondência compatível	Seleccionar esse método
Existem múltiplas correspondências compatíveis	Erro: Reference to method is ambiguous
Nenhuma correspondência	Erro: Cannot find symbol

Tabela 3 — Algoritmo de resolução de sobrecarga de métodos.

3. Geração de Código

3.1 Arquitectura do Gerador

O gerador de código (**codegen.c**) produz LLVM IR directamente para **stdout**, consumindo a AST já anotada semanticamente e as tabelas de símbolos construídas na fase anterior. A geração segue um percurso em profundidade da AST: **gen_expr** processa expressões e devolve o número do registo temporário resultante; **gen_stmt** processa statements e devolve 1 se o fluxo de execução terminar com um **return**, 0 caso contrário. Esta distinção é usada para suprimir saltos desnecessários após blocos que já terminam com **return**.

3.2 Mapeamento de Tipos

Tipo Juc	Tipo LLVM IR	Valor inicial
int	i32	0
boolean	i1	0
double	double	0.0
void	void	—
String[]	i8**	null

Tabela 4 — Mapeamento de tipos entre Juc e LLVM IR.

3.3 Estrutura do Ficheiro LLVM IR

O ficheiro de saída segue uma estrutura fixa, emitida pela função **generate_code**:

```

1. Declarações externas: declare i32 @printf(i8*, ...)
                        declare i32 @atoi(i8*)
2. Constantes de formato: @.fmt.int, @.fmt.double, @.fmt.true, @.fmt.false, @.fmt.str
3. Variáveis globais:   @global_argc = global i32 0
                        @global_argv = global i8** null
4. Literais de string:  @.str.N (recolhidos por find_strings)
5. Variáveis de campo:  @nome = global tipo 0
6. Funções de método:   define @nome_mangled() { ... }
```

3.4 Name Mangling para Suporte a Sobrecarga

Uma vez que LLVM IR não suporta sobrecarga de funções, cada método recebe um nome único que codifica a assinatura dos parâmetros pela função **get_mangled_name**. Os sufixos usados são **_i** (int), **_d** (double), **_b** (boolean) e **_sa** (String[]). Exemplos:

Assinatura Juc	Nome LLVM
foo(int, double)	@foo_i_d
bar(boolean)	@bar_b

Assinatura Juc	Nome LLVM
calc(int)	@calc_i
main(String[])	@juc_main

Tabela 5 — Exemplos de name mangling para métodos Juc.

O método **main** da linguagem Juc recebe tratamento especial: é gerado um stub **@main** com a assinatura da ABI C (**argc**, **argv**), que guarda os argumentos nas variáveis globais **@global_argc** e **@global_argv** e chama **@juc_main**. Esta separação permite que **args.length** e **Integer.parseInt(args[i])** acessem sempre aos argumentos correctos, independentemente da profundidade de chamada.

3.5 Estrutura de uma Função LLVM IR

Cada função segue um esqueleto uniforme. Todas as variáveis locais são alocadas na pilha com **alloca** e inicializadas a zero antes do corpo ser executado. Os parâmetros formais são recebidos com o sufixo **_val** e copiados para a posição na pilha com **store**:

```
define @nome(_val) {
    %return_val = alloca          ; slot de retorno (se não void)
    %logic_res  = alloca i1       ; auxiliar para && e ||
    ; alocação e inicialização de variáveis locais
    ; store _val -> %          ; cópia dos parâmetros
    ; ... corpo de statements ...
    br label %return_end
return_end:
    %tN = load , * %return_val
    ret  %tN
}
```

Os registos temporários são atribuídos sequencialmente (**%t1**, **%t2**, ...) com o contador **tmp_reg**, reiniciado para cada método. Os labels de controlo de fluxo usam o prefixo **L** com o contador **label_cnt**.

3.6 Controlo de Fluxo e Avaliação em Curto-Circuito

if-else: Gera um branch condicional após avaliar a expressão. O bloco **then** e o bloco **else** (se existir) terminam com **br** para o label de saída. Quando ambos os ramos terminam com **return**, o label de saída é suprimido.

while: Gera três labels — **L_cond** (avaliação da condição), **L_loop** (corpo) e **L_end** (saída). O corpo termina com **br label %L_cond**.

&& e || (avaliação em curto-circuito): Implementados com branches condicionais intermédios e a variável auxiliar **%logic_res** na pilha. Para **&&**: se o operando esquerdo for falso, salta directamente para o label de fim (sem avaliar o direito). Para **||**: se for verdadeiro, salta directamente para o fim. O resultado final é carregado de **%logic_res**.


```

; Exemplo para &&:
store i1 %t_left, i1* %logic_res
br i1 %t_left, label %L_true, label %L_end
L_true:
store i1 %t_right, i1* %logic_res
br label %L_end
L_end:
%tN = load i1, i1* %logic_res

```

3.7 Conversões de Tipo e Operações Especiais

A função **gen_expr_cast** envolve a geração de uma subexpressão e, se o tipo do nó for **TY_INT** e o tipo alvo for **TY_DOUBLE**, insere a instrução **sitofp i32 %tN to double**. Esta promoção é invocada em operações aritméticas mistas, comparações e passagem de argumentos.

System.out.print selecciona o formato de impressão conforme o tipo semântico: **printf** com **%d** para int, **%.16e** para double, **%s** para String; para boolean, gera um branch condicional que imprime **"true"** ou **"false"**.

Integer.parseInt(args[i]) incrementa o índice **i** por 1 (o **argv[0]** é o nome do programa), acede a **argv[i+1]** via **getelementptr**, e invoca **@atoi**. **args.length** subtrai 1 ao valor de **@global_argc**.

3.8 Exemplo de Execução — Factorial.java

Para o programa **Factorial.java**, o compilador emite (de forma resumida):

```

declare i32 @printf(i8*, ...)
declare i32 @atoi(i8*)
@.fmt.int = private unnamed_addr constant [3 x i8] c"%d\00"
@global_argc = global i32 0
@global_argv = global i8** null

define i32 @factorial_i(i32 %n_val) {
    %return_val = alloca i32
    %n = alloca i32
    store i32 %n_val, i32* %n
    ; if (n == 0) return 1; return n * factorial(n-1);
    ...
}

define i32 @main(i32 %argc, i8** %argv) {
    store i32 %argc, i32* @global_argc
    store i8** %argv, i8*** @global_argv
    call void @juc_main(i8** %argv)
    ret i32 0
}

```

O ficheiro **.ll** resultante pode ser executado directamente com **lli Factorial.ll 7**, produzindo **5040**.

4. Conclusões e Limitações

O compilador implementa a totalidade das funcionalidades especificadas para as quatro metas do projecto, incluindo: análise léxica com recuperação de erros em strings e comentários; análise sintática com recuperação local e construção da AST sem nós supérfluos; análise semântica com verificação de tipos, resolução de sobrecarga de métodos e detecção de erros de limite numérico; e geração de código LLVM IR completo com suporte a todos os operadores, tipos e estruturas de controlo da linguagem Juc, incluindo avaliação em curto-circuito e promoção automática `int`→`double`.

A principal limitação identificada é a ausência de libertação explícita da memória alocada dinamicamente para nós da AST e tabelas de símbolos, o que constitui uma não-conformidade com o enunciado embora sem impacto prático nos casos de teste. Adicionalmente, a variável `%logic_res` é sempre alocada em cada função, mesmo na ausência de operadores `&&` ou `||`.

Como melhorias futuras, destacam-se a implementação de uma fase de optimização de código IR (propagação de constantes, eliminação de código morto), a libertação correcta de memória, e a extensão do suporte a outros tipos Java como `long` ou `char`.